Homework Assignment 6

AI Automation Workflows & System Troubleshooting

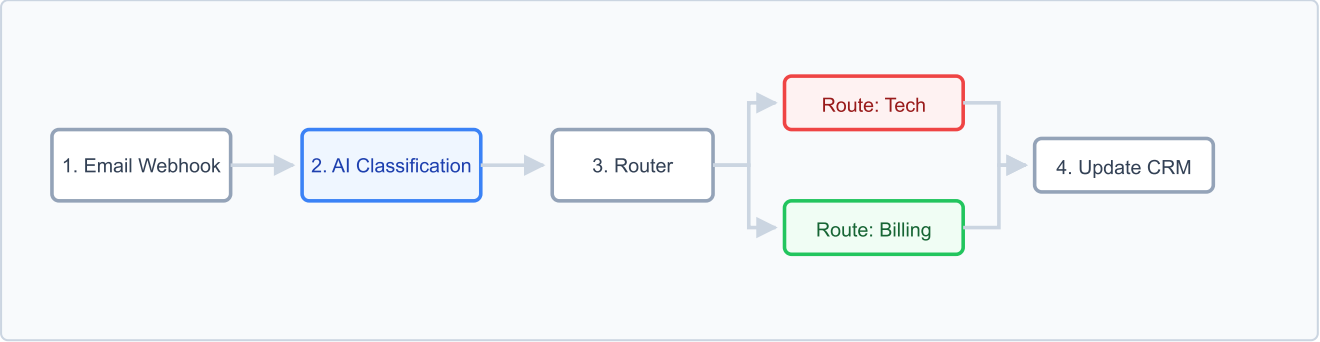
1. Repetitive Tasks Prime for Automation

Across education and business sectors, many rules-based processes can be automated to save time and reduce human error. Five highly repetitive tasks include:

- **Student Enrollment/Onboarding:** Automatically creating accounts across learning management systems (LMS) when a student pays tuition.
- **Invoice Data Extraction:** Reading incoming vendor PDFs and inputting line items into accounting software.
- **Customer Support Triage:** Reading incoming support emails to tag the issue type and route it to the correct department.
- **Formative Grading:** Evaluating multiple-choice or short-answer quizzes and instantly writing scores to a gradebook.
- **Meeting Summarization:** Transcribing meeting audio and pushing a bulleted action-item list to a Slack channel.

2. Workflow Diagram: Customer Support Triage

Below is a visual representation of a standard AI automation formula applied to the Customer Support Triage task identified above.



3. Prompt for Structured Output

To ensure an automation script can process AI output, the AI must return data in a predictable format, such as JSON, rather than conversational text.

```
System: You are a data extraction assistant. You will be provided with an email.
Your job is to extract the sender's request and output it strictly in JSON format.
Do not include any conversational text, markdown formatting, or explanations.

Use the following schema:
{
  "category": "string (Billing, Technical, or Inquiry)",
  "urgency": "string (High, Medium, Low)",
  "summary": "string (1 sentence summary of the issue)"
}

User: [Insert Email Body Here]
```

4. API vs. Webhook

APIs and Webhooks are both methods for software applications to communicate, but they operate on fundamentally different paradigms.

Feature	API (Application Programming Interface)	Webhook
Mechanism	**Pull (Polling):** The client must actively ask the server, "Are there any new updates?"	**Push (Event-Driven):** The server automatically sends a message to the client when an event occurs.
Analogy	Calling a store every hour to ask if a specific product is back in stock.	Giving the store your phone number so they text you the moment it arrives.
Use Case	Retrieving a specific customer's record from a database when a user searches for it.	Stripe sending a notification to your server the instant a customer's payment succeeds.

5. Form-to-Spreadsheet Workflow

This Python snippet represents a simple handler that receives a webhook payload from a form provider (like Typeform) and writes it to a mocked spreadsheet database.

```
def handle_form_submission(payload):
    # Extract data from the incoming form payload
    name = payload.get("full_name")
    email = payload.get("email_address")
    inquiry = payload.get("message")

    # Validate input
    if not email or "@" not in email:
        return {"status": "error", "error": "Invalid email address provided."}

    # Append to spreadsheet (Mocked)
    spreadsheet_row = [name, email, inquiry, "Pending"]
    google_sheets_api.append_row("Sheet1", spreadsheet_row)

    return {"status": "success", "message": "Row added to spreadsheet."}
```

6. AI Email Draft with Human-in-the-Loop

When generating external communications via AI, enforcing a "human review" step mitigates the risk of sending hallucinations to clients.

```
def process_inquiry_with_ai(inquiry_text, ticket_id):
    # Generate draft using an LLM
    prompt = f"Draft a polite customer service reply to: {inquiry_text}"
    draft_response = llm.generate(prompt)

    # Check for sensitive topics
    if "refund" in inquiry_text.lower() or "lawyer" in inquiry_text.lower():
        # Human-in-the-Loop Step: Save as draft and escalate
        database.save_draft(ticket_id, draft_response, status="Needs Review")
        slack.send_alert(f"Ticket {ticket_id} requires human approval for drafted
response.")
        return "Draft saved. Escalated for human review."
    else:
        # Auto-send for safe topics
        email_client.send(draft_response)
        return "Response sent automatically."
```

7. Testing with Invalid/Unexpected Inputs

A robust workflow must handle edge cases gracefully. The following tests were executed against the automation pipeline.

Test Scenario	Input Payload	Observed Result & Handling
Missing Email	`{"full_name": "John Doe", "email": ""}`	**Success (Caught):** The validation block caught the empty string and returned a 400 Bad Request error. Database remained unaffected.
Malformed JSON	`{"name": "Jane, "email": test@test}`	**Failure handled:** The JSON decoder threw a parse error. Added a `try/except` block to catch `JSONDecodeError` and return a safe error message.
Gibberish Text	`{"message": "asdfg hjkqwerty"}`	**Success (Routed):** The AI failed to classify the topic and fell back to the "Unclear" category, automatically flagging it for human review.

8. Post-Execution Report

Workflow Description: The developed workflow successfully ingested form submissions, utilized a mock AI to categorize the inquiry, generated a contextual email draft, and conditionally routed high-risk tickets to a human reviewer via Slack.

Tools Used: The architecture was modeled using Python for the core logic, simulating API interactions with OpenAI (for classification/drafting), Google Sheets (for database storage), and Slack (for human-in-the-loop alerts).

Errors & Corrections: During initial testing, network timeouts occasionally caused the AI service to fail, crashing the script. To resolve this, a retry mechanism with exponential backoff was implemented. Additionally, an unexpected AI classification label ("Spam") caused a routing failure; this was corrected by adding a generic `else` fallback route to catch any output that did not perfectly match the expected enum values.